

Functional Design & Business Logic

Lab & Classroom Resource Booking Portal

Current Development Progress

We have now started the development phase of the project. So far, we have completed the **Faculty Dashboard**, which consists of:

- A main dashboard displaying:
 - Calendar
 - Number of available laboratories
 - Total bookings made by the logged-in faculty
 - Pending booking requests
- Profile Module
- Pending Approvals section for the booking module

This gives the faculty a quick overview of their bookings and the current availability of resources.

Proposed Booking Flow

For the next phase, I was thinking of implementing the **core booking logic** while making the booking process a little more intuitive than a traditional booking form.

Instead of directly asking the faculty to fill in all booking details, the booking process could follow this flow:

1. The faculty selects the required **date** from the calendar on the dashboard.
2. Based on the selected date, the system displays all available resources along with their current status, such as **Available** or **Booked**.
3. The faculty then selects one of the available classrooms or laboratories.
4. Once a resource is selected, the system displays the available **time slots** for that specific resource on the selected date.
5. The faculty selects the preferred time slot and enters the **purpose of the booking** before submitting the request.

6. The system validates the user's authorization, resource eligibility, capacity and current availability before attempting to create the booking.
7. The booking is then processed as a **single atomic database operation** so that the same resource and time slot cannot be successfully allocated to two users at the same time.

This makes the booking process more user-friendly while also ensuring that the availability shown to the user does not become a source of incorrect bookings.

The availability displayed while browsing is only an indication of the current state. The **final database operation** is responsible for deciding whether the booking actually succeeds.

Conflict Handling Logic

While implementing the booking module, one of the important aspects is handling situations where multiple faculty members attempt to book the **same resource at the same time**.

Case 1: Multiple Requests for an Available Resource

If two or more users try to book the same available resource and time slot, the system should not first perform a separate availability check and then decide who gets the resource.

Instead, all competing requests should go through the **same atomic database reservation operation**.

The database will enforce that only one active booking can occupy the same resource and time slot.

Whichever transaction successfully creates the reservation first will obtain the resource, while the other transaction will receive a conflict and its booking will not be created.

This effectively gives us **FCFS behaviour without depending on an arbitrary timestamp threshold**.

The system should use **server-side/database ordering** rather than client device timestamps because client clocks may not be synchronized.

No Separate "Simultaneous Conflict" Case

There should not be a separate category such as:

"If two users book within X milliseconds, send both to Admin."

Server timestamps are always ordered, so defining an artificial threshold would make the system difficult to implement fairly.

Instead, if two users compete for the same free resource and time slot, the **database transaction itself resolves the technical race**.

The user whose transaction successfully obtains the reservation wins.

The other user receives a normal booking conflict.

Admin intervention should **not** be triggered simply because two requests reached the server very close to each other.

This keeps the normal booking process automatic and avoids creating an unnecessary Admin bottleneck.

Business Priority and Admin Approval

Admin approval should be used for **genuine business or institutional priority cases**, rather than being used as a solution to a technical concurrency problem.

The **purpose of the booking should therefore be collected upfront from every user**, before the booking request is submitted.

This ensures that the purpose is available whether or not a conflict occurs.

If a booking requires manual approval according to the project's authorization or business rules, the request can be sent to the Admin.

The Admin can review information such as:

- Faculty Name
- Purpose of Booking
- Requested Resource
- Requested Date and Time
- Required capacity

- Required equipment/facilities
- Relevant priority information

The system should not automatically decide that one purpose is more legitimate than another.

The **Admin makes the final decision** when an actual institutional priority decision is required.

Resource Locking and Concurrency

Resource locking is required only as part of the controlled booking operation and should **never remain active indefinitely**.

The application should not use a sequence such as:

Check availability → Lock resource → Insert booking

as three independent operations.

This can still create a race condition because another transaction can change the resource between those steps.

Instead, the availability validation and reservation must be handled **atomically inside the same database transaction**.

If an application-level or advisory lock is used, it must have a **bounded lifetime** and must be released when:

- The transaction completes
- The transaction fails
- The transaction is rolled back
- The lock times out

This prevents an abandoned request from leaving a resource locked indefinitely.

The lock is only a synchronization mechanism.

The actual correctness of preventing double booking comes from the **database transaction and database-level constraint** on the resource and requested time.

Transaction-Based Booking

The complete booking process should be handled as a **single database transaction** instead of executing independent queries.

The flow would be:

BEGIN TRANSACTION

1. Validate the user's authorization and booking requirements.
2. Validate the requested resource, date and time.
3. Perform the availability/reservation operation atomically.
4. Insert the booking record.
5. Apply the database-level constraint that prevents another active booking from occupying the same resource and time.
6. Commit the transaction if everything succeeds.

COMMIT

If any step fails:

ROLLBACK

If two users try to reserve the same resource and time concurrently, the database must ensure that both transactions cannot successfully commit the same occupancy.

This prevents the classic **check-then-act race condition**.

The transaction should also use an appropriate **isolation/locking strategy** so that concurrent booking attempts cannot both pass an availability check and then commit conflicting bookings.

If the booking operation fails, the user should receive a clear message that:

"This resource is no longer available for the selected time."

There should not be a partially created booking left in the database.

Conflict Detection

When a booking request is made, the system should check for conflicts based on:

- Same Resource
- Same Date
- Overlapping Time Period

Instead of checking only whether the start and end times are exactly equal, the system should check whether the requested intervals overlap.

The time intervals should be treated as **half-open intervals** [start, end).

This means that:

Booking A: 9:00 AM – 10:00 AM

Booking B: 10:00 AM – 11:00 AM

are **not considered overlapping**.

The first booking ends exactly when the second one begins.

If the project requires a setup/cleanup or turnaround period between bookings, that buffer should be considered part of the occupied interval during conflict checking.

For example, if a 15-minute cleanup period is required, the resource may effectively remain unavailable for an additional 15 minutes after the booking.

FCFS Policy

If multiple faculty members try to book the same available resource and time, the system should follow an automatic **First Come, First Served (FCFS)** approach for bookings that are eligible for automatic allocation.

However, FCFS should **not** be implemented by simply comparing timestamps after separate availability checks.

The booking operation itself must be atomic.

The first transaction that successfully obtains the database reservation wins.

A competing transaction attempting to reserve the same resource and time will fail the database constraint and will be handled as a normal booking conflict.

Server-side/database ordering should be used instead of the user's device time because client clocks may differ.

This removes the need for a separate:

"simultaneous booking"

condition.

Alternative Resource Suggestion

If a user's booking request cannot be completed because the requested resource is already occupied, the system should not simply display:

Booking Rejected

Instead, the system should search for other resources that are suitable for the **same date and requested time**.

The alternatives should be selected using actual resource attributes available in the system rather than an undefined idea of "similar resource."

The matching can consider factors such as:

- Same resource type
- Required capacity
- Required equipment or facilities
- Location, where relevant
- Department or resource restrictions, where applicable
- Availability for the complete requested time

For example, if a faculty member requested a laboratory for a group of 40 students, the system should not suggest a laboratory whose capacity is only 20.

The suggested alternative is only an available option at the time it is displayed.

It is **not automatically reserved** for the user.

When the user selects the suggested alternative, the system must run the **same atomic booking operation again**.

This is important because another faculty member may have booked that alternative resource between the time it was displayed and the time the user selected it.

Therefore:

Show Alternative → User Selects Alternative → Revalidate → Attempt Atomic Booking

Alternative resources should be offered to **any booking request that loses an automatic allocation**, not only to requests that go through Admin approval.

Queue for Pending Approvals

Whenever a booking genuinely requires manual approval from the Admin, the request should be placed in an approval queue.

However, because Admin decisions may depend on institutional priority rather than simply arrival order, this should be treated as a **Priority Queue**, not a normal FIFO queue.

The requests can be ordered using the defined priority of the request/user, with request time being used as a tie-breaker where appropriate.

This is different from the technical FCFS booking race.

FCFS

Used for an automatically allocatable resource where multiple users compete for the same available slot.

Priority Queue

Used for requests that genuinely require Admin review.

Therefore, the two concepts should not be mixed.

Priority Resolution

When a booking requires Admin review, the system should provide all the necessary information to support the decision-making process.

This information can include:

- Faculty Name
- Purpose of Booking
- Requested Resource
- Requested Date and Time
- Required Capacity
- Required Facilities/Equipment
- Relevant Booking Priority

The system itself should **not decide which purpose is more legitimate**.

Instead, it presents the relevant information so that the Admin can make an informed decision based on institutional priorities and requirements.

If an Admin overrides the normal priority, the reason for that decision should also be recorded.

Cancellation and Modification

The booking module should also define what happens when an existing booking is **cancelled or modified**.

For cancellation, the system should first verify that the user is authorized to cancel the booking.

The booking status should then be changed to:

Cancelled

and the corresponding resource/time allocation should be released.

The cancellation should also be recorded in the booking history/audit trail.

If the released slot can now be used by another pending or interested request, the system can notify the relevant users according to the notification rules.

Modification

Modification should not bypass the normal conflict checks.

For example, if a faculty member wants to change:

- Resource
- Date
- Start time
- End time

the new request must go through the same authorization and availability validation.

If the new slot cannot be obtained, the **original booking should remain unchanged**.

This prevents a failed modification from accidentally cancelling an already valid booking.

No-Show and Auto-Release Policy

The system should also define what happens when a confirmed booking is not actually used.

If the project includes a no-show rule, the system can mark a booking as **Expired** after the defined condition is reached and release the occupied resource.

However, the release condition and timing must be explicitly defined.

The system should not claim to automatically detect a physical no-show unless there is an actual attendance or resource-usage mechanism available to provide that information.

Once a booking is cancelled or auto-released, the resource becomes available again.

Relevant users can then be notified if required.

Recurring Bookings

The booking module should support **recurring bookings**, since faculty may need the same laboratory or classroom repeatedly.

For example:

Every Monday from 10:00 AM – 12:00 PM for one semester.

For a recurring booking, the faculty member selects:

- Recurrence pattern
- Start date
- End date
- Time
- Resource

The system should then generate the individual booking occurrences and validate **each occurrence separately**.

If one particular date is already occupied, that occurrence should not overwrite the existing booking.

The user should be shown which occurrences were successfully booked and which occurrences could not be booked because of conflicts or other restrictions.

Authorization and Resource Eligibility

The system should not assume that every faculty member can book every resource.

Before creating a booking, the system should verify whether the faculty member has permission to use the selected resource according to the defined role and resource rules.

Where applicable, the system should also check:

- Department restrictions
- Resource restrictions
- Capacity
- Required facilities/equipment

For example, if a classroom has a capacity of 30 students, a booking for a group of 50 should not be accepted for that classroom.

Similarly, if a laboratory is restricted to a particular department or type of activity, an unauthorized faculty member should not be able to book it.

Admin Audit Trail and Concurrent Admin Actions

All important Admin actions should be recorded so that there is a clear history of what happened to a booking request.

This should include actions such as:

- Approval
- Rejection
- Priority override
- Cancellation
- Resource allocation changes

The system must also handle the situation where **two Admins open and attempt to process the same request at the same time**.

The status update should be atomic so that only one valid final decision is applied.

For example:

Admin A → Approves request

Admin B → Attempts to approve/reject the same request

The system should detect that the request has already been processed and prevent Admin B from overwriting Admin A's decision.

The second Admin should instead see that the request has already been processed.

Where an Admin overrides the normal priority or allocation, the reason for the override should be recorded in the audit trail.

Notification Reliability

Notifications should be treated as a way of **informing the user**, not as the source of truth for booking status.

If a user does not see a notification, the booking status should still remain correctly stored in the system.

The user should be able to see the status through:

- Dashboard
- Pending Requests
- Booking History

Notifications can have statuses such as:

- Pending
- Sent
- Failed
- Read

where applicable.

If a notification fails, it should be possible to retry it.

A notification should also **not be responsible for holding a resource**.

Resource occupancy is controlled by the database booking state.

Operating Systems Concepts Used

The booking module can demonstrate Operating Systems concepts through concurrent access to shared resources.

1. Critical Section

The reservation operation modifies shared resource availability and therefore needs to be protected from unsafe concurrent updates.

2. Synchronization

Concurrent booking requests must be coordinated so that two users cannot successfully reserve the same resource and time.

3. Mutex / Locking

A database or advisory lock can be used as a synchronization mechanism.

However, the mutex should **not be considered the final correctness mechanism**.

It must have a bounded lifetime and must be released when the transaction completes, fails or times out.

4. Atomicity

The booking operation should either complete successfully or be rolled back completely.

There should not be a situation where:

Booking record created → availability update fails

and the system is left in an inconsistent state.

5. Deadlock Prevention

Locks must not be held indefinitely.

They should be released when the transaction completes, fails or times out.

6. Concurrency Control

Multiple faculty members can attempt bookings at the same time while the database maintains a consistent final state.

The important point is:

The mutex/lock demonstrates synchronization, but the database transaction and database-level constraint provide the actual guarantee that conflicting bookings cannot both be committed.

Data Structures Used

1. Priority Queue

A **Priority Queue** can be used for booking requests that genuinely require Admin approval, where priority determines the order of review.

2. Queue

A normal Queue can be used separately for things such as:

- Notification delivery

- Background processing
- Other sequential tasks

The normal Queue and Priority Queue should not be treated as the same thing.

A FIFO queue would process requests strictly in arrival order, whereas Admin approval may require priority-based processing.

3. Database Indexes

Indexes can be used to make searches faster for:

- Resource availability
- Booking history
- Pending approvals
- User bookings

4. Resource Matching

Resource attributes such as:

- Type
- Capacity
- Equipment
- Location

can be used to search for suitable alternative resources.

Overall Booking Logic

The overall logic can therefore be summarized as follows:

1. **Faculty selects date** → system shows resources.
2. **Faculty selects resource** → system shows available time slots.
3. **Faculty selects time** → system collects the booking purpose and required details.
4. **Server validates** → authorization, capacity and resource rules.
5. **Booking is attempted** → through one atomic database operation.
6. **If the resource/time is still free** → transaction succeeds and booking is created.

7. **If another transaction has already occupied the same resource/time** → booking fails safely as a conflict.
8. There is **no artificial "simultaneous conflict" threshold**.
9. There is **no automatic Admin escalation simply because two users attempted the booking at nearly the same time**.
10. If the request genuinely requires institutional approval → it enters the **priority-based Admin approval workflow**.
11. If a booking is unsuccessful → suitable alternative resources can be suggested.
12. When an alternative is selected → it is **revalidated through the same atomic booking operation**.
13. Cancellation and modification follow the same authorization and conflict rules.
14. Recurring bookings validate each occurrence separately.
15. Expired/cancelled bookings release their resource allocation according to the defined rules.
16. Important booking and Admin state changes are recorded for **history and audit**.
17. Notifications inform users of changes, but the **database remains the source of truth**.